

PLC Design in TwinCAT 3



- **Same machine** — three architectures.
- Same three change requests, every stage.
- **One running scoreboard** — nine cells.
- When does each architecture pay off?

- Block 0 · Welcome · 5 min
- **Block 1 · Stage 1** — Procedural · 45 min
- Break · 15 min
- **Block 2 · Stage 2** — Inheritance · 60 min
- Break · 15 min
- **Block 3 · Stage 3** — Composition · 90 min
- Block 4 · Scoreboard + take-home · 10 min



Who here has helped ship a 4-station-or-larger machine in the last year?

— Then this workshop is calibrated for you.

Stage 1 — Procedural

45 min

The architecture you've been writing for 20 years.

- **Origin** — IEC 61131-3 (1993).
- **Mental model** — FB has its own independent version of code, runs once per cycle.
- **Good at simple machines** — short lifespan, no changes to code, single developer
- **Bad at change requests** — can require edits to every FB file, requires intense discipline across time and team

Goal:

- Get familiar with the procedural sample structure
- Understand switching between branches
- Identify similarities and differences in the function block stations

Instructions:

- Open the release branch and go to the FillingLine_Procedural Project
- Open the four Function Block stations in the POU's folder
- Stack the open FBs side by side

Every station has the same five sections

BECKHOFF

- **Mode block** — read ModeAuto / ModeManual / ModeStop.
- **Alarm-ack block** — clear latched alarms.
- **Reset block** — return state to 0.
- **HMI mapping** — copy state to HMI tags.
- **State machine** — the actual sequence (CASE).

Every station has the same five sections

BECKHOFF

- Mode block — read ModeAuto / ModeManual / ModeStop.

```
FB_StationCap
1 FUNCTION_BLOCK FB_StationCap
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  CapFeederReady : BOOL;
12  TorqueLimit   : LREAL;
13 END_VAR
14 VAR_OUTPUT
15   Busy : BOOL;
16 END_VAR
17 // mode
18 IF ModeAuto THEN
19   ActiveMode := 1;
20 ELSEIF ModeManual THEN
21   ActiveMode := 2;
22 ELSE
23   ActiveMode := 0;
24 END_IF
25 // alarm ack
26 IF AlarmLatched AND AlarmAck THEN
27   AlarmLatched := FALSE;
28   AlarmCode := 0;
29   AlarmText := '';
30 END_IF
31 // Reset
32 IF Reset THEN
33   State := 0;
34   AlarmLatched := FALSE;
35   AlarmCode := 0;
36   AlarmText := '';
37   Timeout(IN := FALSE);
38   CapSettle(IN := FALSE);
39   Done := FALSE;
40 END_IF
41 // HMI Mapping
42 StateDisplay := State;
43 CASE State OF
44   0: StatusText := 'Idle';
45   10: StatusText := 'Pick Cap';
46   20: StatusText := 'Torquing';
47   30: StatusText := 'Complete';
48 ELSE
49   StatusText := 'Error';
50 END_CASE
51 END_FUNCTION_BLOCK

FB_StationFill
1 FUNCTION_BLOCK FB_StationFill
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  FlowRate      : LREAL;
12  TargetVolume  : LREAL;
13 END_VAR
14 VAR_OUTPUT
15   Busy : BOOL;
16 END_VAR
17 // mode
18 IF ModeAuto THEN
19   ActiveMode := 1;
20 ELSEIF ModeManual THEN
21   ActiveMode := 2;
22 ELSE
23   ActiveMode := 0;
24 END_IF
25 // alarm ack
26 IF AlarmLatched AND AlarmAckEdge.Q THEN
27   AlarmLatched := FALSE;
28   AlarmCode := 0;
29   AlarmText := '';
30 END_IF
31 // Reset
32 IF Reset THEN
33   State := 0;
34   AlarmLatched := FALSE;
35   AlarmCode := 0;
36   AlarmText := '';
37   Timeout(IN := FALSE);
38   Accumulated := 0.0;
39   Done := FALSE;
40 END_IF
41 // HMI Mapping
42 StateDisplay := State;
43 CASE State OF
44   0: StatusText := 'Idle';
45   10: StatusText := 'Filling';
46   20: StatusText := 'Complete';
47 ELSE
48   StatusText := 'Error';
49 END_CASE
50 END_FUNCTION_BLOCK

FB_StationInspect
1 FUNCTION_BLOCK FB_StationInspect
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  CameraReady   : BOOL;
12  InspectResult : BOOL;
13 END_VAR
14 VAR_OUTPUT
15   Busy : BOOL;
16 END_VAR
17 // mode
18 IF ModeAuto THEN
19   ActiveMode := 1;
20 ELSEIF ModeManual THEN
21   ActiveMode := 2;
22 ELSE
23   ActiveMode := 0;
24 END_IF
25 // alarm ack
26 IF AlarmLatched AND AlarmAck THEN
27   AlarmLatched := FALSE;
28   AlarmCode := 0;
29   AlarmText := '';
30 END_IF
31 // Reset
32 IF Reset THEN
33   State := 0;
34   AlarmLatched := FALSE;
35   AlarmCode := 0;
36   AlarmText := '';
37   Timeout(IN := FALSE);
38   CameraDelay(IN := FALSE);
39   Done := FALSE;
40 END_IF
41 // HMI Display
42 StateDisplay := State;
43 CASE State OF
44   0: StatusText := 'Idle';
45   10: StatusText := 'Inspecting';
46   20: StatusText := 'Evaluate';
47   25: StatusText := 'Rejecting';
48   30: StatusText := 'Complete';
49 ELSE
50   StatusText := 'Error';
51 END_CASE
52 END_FUNCTION_BLOCK

FB_StationLabel
1 FUNCTION_BLOCK FB_StationLabel
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  LabelReady    : BOOL;
12  ApplyPressure : LREAL;
13 END_VAR
14 VAR_OUTPUT
15   Busy : BOOL;
16 END_VAR
17 // mode
18 IF ModeAuto THEN
19   ActiveMode := 1;
20 ELSEIF ModeManual THEN
21   ActiveMode := 2;
22 ELSE
23   ActiveMode := 0;
24 END_IF
25 // alarm ack
26 IF AlarmLatched AND AlarmAckEdge.Q THEN
27   AlarmLatched := FALSE;
28   AlarmCode := 0;
29   AlarmText := '';
30 END_IF
31 // Reset
32 IF Reset THEN
33   State := 0;
34   AlarmLatched := FALSE;
35   AlarmCode := 0;
36   AlarmText := '';
37   Timeout(IN := FALSE);
38   ApplySettle(IN := FALSE);
39   Done := FALSE;
40 END_IF
41 // HMI Mapping
42 StateDisplay := State;
43 CASE State OF
44   0: StatusText := 'Idle';
45   100: StatusText := 'Picking Label';
46   200: StatusText := 'Applying';
47   300: StatusText := 'Complete';
48 ELSE
49   StatusText := 'Error';
50 END_CASE
51 END_FUNCTION_BLOCK
```

Every station has the same five sections

BECKHOFF

- ▶ Alarm-ack block — clear latched alarms.

```
FUNCTION_BLOCK FB_StationCap
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    CapFeederReady : BOOL;
    TorqueLimit  : LREAL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// mode
IF ModeAuto THEN
    ActiveMode := 1;
ELSIF ModeManual THEN
    ActiveMode := 2;
ELSE
    ActiveMode := 0;
END_IF
// alarm ack
IF AlarmLatched AND AlarmAck THEN
    AlarmAckEdge(CLK := AlarmAck);
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
END_IF
// Reset
IF Reset THEN
    State := 0;
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
    Timeout(IN := FALSE);
    CapSettle(IN := FALSE);
    Done := FALSE;
END_IF
// HMI Mapping
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    10: StatusText := 'Pick Cap';
    20: StatusText := 'Torquing';
    30: StatusText := 'Complete';
ELSE
    StatusText := 'Complete';
END_CASE
```

```
FUNCTION_BLOCK FB_StationFill
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    FlowRate     : LREAL;
    TargetVolume : LREAL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// mode
IF ModeAuto THEN
    ActiveMode := 1;
ELSIF ModeManual THEN
    ActiveMode := 2;
ELSE
    ActiveMode := 0;
END_IF
// alarm ack
IF AlarmLatched AND AlarmAck THEN
    AlarmAckEdge(CLK := AlarmAck);
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
END_IF
// Reset
IF Reset THEN
    State := 0;
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
    Timeout(IN := FALSE);
    Accumulated := 0.0;
    Done := FALSE;
END_IF
// HMI Mapping
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    10: StatusText := 'Filling';
    20: StatusText := 'Complete';
ELSE
    StatusText := 'Complete';
END_CASE
```

```
FUNCTION_BLOCK FB_StationInspect
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    CameraReady  : BOOL;
    InspectResult : BOOL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// mode
IF ModeAuto THEN
    ActiveMode := 1;
ELSIF ModeManual THEN
    ActiveMode := 2;
ELSE
    ActiveMode := 0;
END_IF
// alarm ack
IF AlarmLatched AND AlarmAck THEN
    AlarmAckEdge(CLK := AlarmAck);
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
END_IF
// Reset
IF Reset THEN
    State := 0;
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
    Timeout(IN := FALSE);
    CameraDelay(IN := FALSE);
    Done := FALSE;
END_IF
// HMI Display
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    10: StatusText := 'Inspecting';
    20: StatusText := 'Evaluate';
    25: StatusText := 'Rejecting';
    30: StatusText := 'Complete';
END_CASE
```

```
FUNCTION_BLOCK FB_StationLabel
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    LabelReady   : BOOL;
    ApplyPressure : LREAL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// mode
IF ModeAuto THEN
    ActiveMode := 1;
ELSIF ModeManual THEN
    ActiveMode := 2;
ELSE
    ActiveMode := 0;
END_IF
// alarm ack
IF AlarmLatched AND AlarmAck THEN
    AlarmAckEdge(CLK := AlarmAck);
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
END_IF
// Reset
IF Reset THEN
    State := 0;
    AlarmLatched := FALSE;
    AlarmCode := 0;
    AlarmText := '';
    Timeout(IN := FALSE);
    ApplySettle(IN := FALSE);
    Done := FALSE;
END_IF
// HMI Mapping
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    100: StatusText := 'Picking Label';
    200: StatusText := 'Applying';
    300: StatusText := 'Complete';
END_CASE
```

Every station has the same five sections

BECKHOFF

- ▶ Reset block — return state to 0.

```
FB_StationCap
1 FUNCTION_BLOCK FB_StationCap
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  CapFeederReady : BOOL;
12  TorqueLimit   : LREAL;
13 END_VAR
14 VAR_OUTPUT
15  Busy          : BOOL;
100 %

1 // mode
2 IF ModeAuto THEN
3   ActiveMode := 1;
4 ELSIF ModeManual THEN
5   ActiveMode := 2;
6 ELSE
7   ActiveMode := 0;
8 END_IF
9
10 // alarm ack
11 IF AlarmLatched AND AlarmAck THEN
12   AlarmLatched := FALSE;
13   AlarmCode := 0;
14   AlarmText := '';
15 END_IF
16
17 // Reset
18 IF Reset THEN
19   State := 0;
20   AlarmLatched := FALSE;
21   AlarmCode := 0;
22   AlarmText := '';
23   Timeout(IN := FALSE);
24   CapSettle(IN := FALSE);
25   Done := FALSE;
26 END_IF
27
28 // HMI Mapping
29 StateDisplay := State;
30 CASE State OF
31   0: StatusText := 'Idle';
32   10: StatusText := 'Pick Cap';
33   20: StatusText := 'Torquing';
34   30: StatusText := 'Complete';
35 ELSE
36   StatusText := 'Complete';
37 END_CASE

FB_StationFill
1 FUNCTION_BLOCK FB_StationFill
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  FlowRate      : LREAL;
12  TargetVolume  : LREAL;
13 END_VAR
14 VAR_OUTPUT
15  Busy          : BOOL;
100 %

1 // mode
2 IF ModeAuto THEN
3   ActiveMode := 1;
4 ELSIF ModeManual THEN
5   ActiveMode := 2;
6 ELSE
7   ActiveMode := 0;
8 END_IF
9
10 // alarm ack
11 IF AlarmAckEdge(CLR := AlarmAck);
12 IF AlarmLatched AND AlarmAckEdge.Q THEN
13   AlarmLatched := FALSE;
14   AlarmCode := 0;
15   AlarmText := '';
16 END_IF
17
18 // Reset
19 IF Reset THEN
20   State := 0;
21   AlarmLatched := FALSE;
22   AlarmCode := 0;
23   AlarmText := '';
24   Timeout(IN := FALSE);
25   Accumulated := 0.0;
26   Done := FALSE;
27 END_IF
28
29 // HMI Mapping
30 StateDisplay := State;
31 CASE State OF
32   0: StatusText := 'Idle';
33   10: StatusText := 'Filling';
34   20: StatusText := 'Complete';
35 ELSE
36   StatusText := 'Complete';
37 END_CASE

FB_StationInspect
1 FUNCTION_BLOCK FB_StationInspect
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  CameraReady   : BOOL;
12  InspectResult : BOOL;
13 END_VAR
14 VAR_OUTPUT
15  Busy          : BOOL;
100 %

1 // mode
2 IF ModeAuto THEN
3   ActiveMode := 1;
4 ELSIF ModeManual THEN
5   ActiveMode := 2;
6 ELSE
7   ActiveMode := 0;
8 END_IF
9
10 // alarm ack
11 IF AlarmLatched AND AlarmAck THEN
12   AlarmLatched := FALSE;
13   AlarmCode := 0;
14   AlarmText := '';
15 END_IF
16
17 // Reset
18 IF Reset THEN
19   State := 0;
20   AlarmLatched := FALSE;
21   AlarmCode := 0;
22   AlarmText := '';
23   Timeout(IN := FALSE);
24   CameraDelay(IN := FALSE);
25   Done := FALSE;
26 END_IF
27
28 // HMI Display
29 StateDisplay := State;
30 CASE State OF
31   0: StatusText := 'Idle';
32   10: StatusText := 'Inspecting';
33   20: StatusText := 'Evaluate';
34   25: StatusText := 'Rejecting';
35   30: StatusText := 'Complete';
36 ELSE
37   StatusText := 'Complete';
38 END_CASE

FB_StationLabel
1 FUNCTION_BLOCK FB_StationLabel
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  LabelReady    : BOOL;
12  ApplyPressure : LREAL;
13 END_VAR
14 VAR_OUTPUT
15  Busy          : BOOL;
100 %

1 // mode
2 IF ModeAuto THEN
3   ActiveMode := 1;
4 ELSIF ModeManual THEN
5   ActiveMode := 2;
6 ELSE
7   ActiveMode := 0;
8 END_IF
9
10 // alarm ack
11 IF AlarmAckEdge(CLR := AlarmAck);
12 IF AlarmLatched AND AlarmAckEdge.Q THEN
13   AlarmLatched := FALSE;
14   AlarmCode := 0;
15   AlarmText := '';
16 END_IF
17
18 // Reset
19 IF Reset THEN
20   State := 0;
21   AlarmLatched := FALSE;
22   AlarmCode := 0;
23   AlarmText := '';
24   Timeout(IN := FALSE);
25   ApplySettle(IN := FALSE);
26   Done := FALSE;
27 END_IF
28
29 // HMI Mapping
30 StateDisplay := State;
31 CASE State OF
32   0: StatusText := 'Idle';
33   100: StatusText := 'Picking Label';
34   200: StatusText := 'Applying';
35   300: StatusText := 'Complete';
36 ELSE
37   StatusText := 'Complete';
38 END_CASE
```

Every station has the same five sections

BECKHOFF

- HMI mapping — copy state to HMI tags.

```
FUNCTION_BLOCK FB_StationCap
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    CapFeederReady : BOOL;
    TorqueLimit  : LREAL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// HMI Mapping
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    10: StatusText := 'Pick Cap';
    20: StatusText := 'Torquing';
    30: StatusText := 'Complete';
ELSE
    StatusText := 'Unknown';
END_CASE
Busy := (State <> 0) AND NOT Done;
AlarmActive := AlarmLatched;
Error := AlarmLatched;
ErrorID := AlarmCode;
// State machine
CASE State OF
    0:
        RunCapMotor := FALSE;
        CapPickCylinder := FALSE;
        Done := FALSE;
        IF Execute AND PartPresent AND CapFeederReady THEN
            State := 10;
        END_IF
    10:
        CapPickCylinder := TRUE;
        CapSettle(IN := TRUE, PT := T#500MS);
        IF CapSettle.Q THEN
            CapSettle(IN := FALSE);
            State := 20;
        END_IF
    20:
        RunCapMotor := TRUE;
        Timeout(IN := TRUE, PT := T#5S);
        IF Timeout.Q THEN
            AlarmLatched := TRUE;
            AlarmCode := 1000;
            AlarmText := 'Fill: Timeout';
        END_IF
    30:
        OpenFillValve := TRUE;
        Timeout(IN := TRUE, PT := T#10S);
        IF Timeout.Q THEN
            AlarmLatched := TRUE;
            AlarmCode := 1001;
            AlarmText := 'Fill: No flow';
        END_IF
END_CASE
```

```
FUNCTION_BLOCK FB_StationFill
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    FlowRate     : LREAL;
    TargetVolume : LREAL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// HMI Mapping
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    10: StatusText := 'Filling';
    20: StatusText := 'Complete';
ELSE
    StatusText := 'Unknown';
END_CASE
Busy := (State <> 0) AND NOT Done;
AlarmActive := AlarmLatched;
Error := AlarmLatched;
ErrorID := AlarmCode;
// State machine
CASE State OF
    0:
        OpenFillValve := FALSE;
        Done := FALSE;
        Accumulated := 0.0;
        IF Execute AND PartPresent AND ActiveMode = 1 THEN
            State := 10;
        END_IF
    10:
        OpenFillValve := TRUE;
        Timeout(IN := TRUE, PT := T#10S);
        IF Timeout.Q THEN
            AlarmLatched := TRUE;
            AlarmCode := 1000;
            AlarmText := 'Fill: Timeout';
        END_IF
    20:
        IF FlowRate < 0.1 THEN
            AlarmLatched := TRUE;
            AlarmCode := 1001;
            AlarmText := 'Fill: No flow';
        END_IF
END_CASE
```

```
FUNCTION_BLOCK FB_StationInspect
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    CameraReady  : BOOL;
    InspectResult : BOOL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// HMI Display
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    10: StatusText := 'Inspecting';
    20: StatusText := 'Evaluate';
    25: StatusText := 'Rejecting';
    30: StatusText := 'Complete';
ELSE
    StatusText := 'Unknown';
END_CASE
Busy := (State <> 0) AND NOT Done;
AlarmActive := AlarmLatched;
Error := AlarmLatched;
ErrorID := AlarmCode;
// State machine
CASE State OF
    0:
        TriggerCamera := FALSE;
        RejectPusher := FALSE;
        PassResult := FALSE;
        Done := FALSE;
        IF Execute AND PartPresent AND CameraReady THEN
            State := 10;
        END_IF
    10:
        TriggerCamera := TRUE;
        CameraDelay(IN := TRUE, PT := T#200MS);
        Timeout(IN := TRUE, PT := T#2S);
        IF Timeout.Q THEN
            AlarmLatched := TRUE;
            AlarmCode := 4001;
            AlarmText := 'Inspect: Camera timeout';
        END_IF
        IF CameraDelay.Q THEN
            State := 20;
        END_IF
    20:
        ExtendApplicator := TRUE;
        ApplySettle(IN := TRUE, PT := T#400MS);
        Timeout(IN := TRUE, PT := T#3S);
        IF Timeout.Q THEN
            State := 30;
        END_IF
    30:
        StatusText := 'Complete';
END_CASE
```

```
FUNCTION_BLOCK FB_StationLabel
VAR_INPUT
    Execute      : BOOL;
    Reset        : BOOL;
    ModeAuto     : BOOL;
    ModeManual   : BOOL;
    JogForward   : BOOL;
    JogReverse   : BOOL;
    PartPresent  : BOOL;
    AlarmAck     : BOOL;
    LabelReady   : BOOL;
    ApplyPressure : LREAL;
END_VAR
VAR_OUTPUT
    Busy : BOOL;
// HMI Mapping
StateDisplay := State;
CASE State OF
    0: StatusText := 'Idle';
    100: StatusText := 'Picking Label';
    200: StatusText := 'Applying';
    300: StatusText := 'Complete';
ELSE
    StatusText := 'Unknown';
END_CASE
Busy := (State <> 0) AND NOT Done;
AlarmActive := AlarmLatched;
Error := AlarmLatched;
ErrorID := AlarmCode;
// State machine
CASE State OF
    0:
        LabelVacuum := FALSE;
        ExtendApplicator := FALSE;
        Done := FALSE;
        IF Execute AND PartPresent AND LabelReady AND ActiveMode = 1 THEN
            State := 100;
        END_IF
    100:
        LabelVacuum := TRUE;
        ApplySettle(IN := TRUE, PT := T#300MS);
        IF ApplySettle.Q THEN
            State := 200;
        END_IF
    200:
        ExtendApplicator := TRUE;
        ApplySettle(IN := TRUE, PT := T#400MS);
        Timeout(IN := TRUE, PT := T#3S);
        IF Timeout.Q THEN
            State := 300;
        END_IF
    300:
        StatusText := 'Complete';
END_CASE
```

Every station has the same five sections

BECKHOFF

- State machine — the actual sequence (CASE).

```
1 FUNCTION_BLOCK FB_StationCap
2 VAR_INPUT
3   Execute      : BOOL;
4   Reset        : BOOL;
5   ModeAuto     : BOOL;
6   ModeManual   : BOOL;
7   JogForward   : BOOL;
8   JogReverse   : BOOL;
9   PartPresent  : BOOL;
10  AlarmAck      : BOOL;
11  CapFeederReady : BOOL;
12  TorqueLimit   : LREAL;
13 END_VAR
14 VAR_OUTPUT
15  Busy          : BOOL;
16
17 // HMI Mapping
18 StateDisplay := State;
19 CASE State OF
20   0: StatusText := 'Idle';
21   10: StatusText := 'Pick Cap';
22   20: StatusText := 'Torquing';
23   30: StatusText := 'Complete';
24 ELSE
25   StatusText := 'Unknown';
26 END_CASE
27 Busy := (State <> 0) AND NOT Done;
28 AlarmActive := AlarmLatched;
29 Error := AlarmLatched;
30 ErrorID := AlarmCode;
31
32 // State machine
33 CASE State OF
34   0:
35     RunCapMotor := FALSE;
36     CapPickCylinder := FALSE;
37     Done := FALSE;
38     Accumulated := 0.0;
39     IF Execute AND PartPresent AND CapFeederReady
40       State := 10;
41     END_IF
42   10:
43     CapPickCylinder := TRUE;
44     CapSettle(IN := TRUE, PT := T#500MS);
45     IF CapSettle.Q THEN
46       CapSettle(IN := FALSE);
47       State := 20;
48     END_IF
49   20:
50     RunCapMotor := TRUE;
51     Timeout(IN := TRUE, PT := T#5S);
52     IF Timeout.Q THEN
53       State := 30;
54     END_IF
55   30:
56     State := 0;
57   END_CASE
58
59 FB_StationFill
60
61 FUNCTION_BLOCK FB_StationFill
62 VAR_INPUT
63   Execute      : BOOL;
64   Reset        : BOOL;
65   ModeAuto     : BOOL;
66   ModeManual   : BOOL;
67   JogForward   : BOOL;
68   JogReverse   : BOOL;
69   PartPresent  : BOOL;
70   AlarmAck      : BOOL;
71   FlowRate     : LREAL;
72   TargetVolume : LREAL;
73 END_VAR
74 VAR_OUTPUT
75  Busy          : BOOL;
76
77 // HMI Mapping
78 StateDisplay := State;
79 CASE State OF
80   0: StatusText := 'Idle';
81   10: StatusText := 'Filling';
82   20: StatusText := 'Complete';
83 ELSE
84   StatusText := 'Unknown';
85 END_CASE
86 Busy := (State <> 0) AND NOT Done;
87 AlarmActive := AlarmLatched;
88 Error := AlarmLatched;
89 ErrorID := AlarmCode;
90
91 // State machine
92 CASE State OF
93   0:
94     OpenFillValve := FALSE;
95     Done := FALSE;
96     Accumulated := 0.0;
97     IF Execute AND PartPresent AND ActiveMode =
98       State := 10;
99     END_IF
100   10:
101     OpenFillValve := TRUE;
102     Timeout(IN := TRUE, PT := T#10S);
103     IF Timeout.Q THEN
104       AlarmLatched := TRUE;
105       AlarmCode := 1000;
106       AlarmText := 'Fill: Timeout';
107     END_IF
108   20:
109     IF FlowRate < 0.1 THEN
110       AlarmLatched := TRUE;
111       AlarmCode := 1001;
112       AlarmText := 'Fill: No flow';
113     END_IF
114   30:
115     State := 0;
116   END_CASE
117
118 FB_StationInspect
119
120 FUNCTION_BLOCK FB_StationInspect
121 VAR_INPUT
122   Execute      : BOOL;
123   Reset        : BOOL;
124   ModeAuto     : BOOL;
125   ModeManual   : BOOL;
126   JogForward   : BOOL;
127   JogReverse   : BOOL;
128   PartPresent  : BOOL;
129   AlarmAck      : BOOL;
130   CameraReady   : BOOL;
131   InspectResult : BOOL;
132 END_VAR
133 VAR_OUTPUT
134  Busy          : BOOL;
135
136 // HMI Display
137 StateDisplay := State;
138 CASE State OF
139   0: StatusText := 'Idle';
140   10: StatusText := 'Inspecting';
141   20: StatusText := 'Evaluate';
142   25: StatusText := 'Rejecting';
143   30: StatusText := 'Complete';
144 ELSE
145   StatusText := 'Unknown';
146 END_CASE
147 Busy := (State <> 0) AND NOT Done;
148 AlarmActive := AlarmLatched;
149 Error := AlarmLatched;
150 ErrorID := AlarmCode;
151
152 // State machine
153 CASE State OF
154   0:
155     TriggerCamera := FALSE;
156     RejectPusher := FALSE;
157     PassResult := FALSE;
158     Done := FALSE;
159     IF Execute AND PartPresent AND CameraReady
160       State := 10;
161     END_IF
162   10:
163     TriggerCamera := TRUE;
164     CameraDelay(IN := TRUE, PT := T#200MS);
165     Timeout(IN := TRUE, PT := T#2S);
166     IF Timeout.Q THEN
167       AlarmLatched := TRUE;
168       AlarmCode := 4001;
169       AlarmText := 'Inspect: Camera timeout';
170     END_IF
171   20:
172     IF CameraDelay.Q THEN
173       RejectPusher := TRUE;
174       State := 25;
175     END_IF
176   25:
177     IF RejectPusher.Q THEN
178       PassResult := TRUE;
179       State := 30;
180     END_IF
181   30:
182     State := 0;
183   END_CASE
184
185 FB_StationLabel
186
187 FUNCTION_BLOCK FB_StationLabel
188 VAR_INPUT
189   Execute      : BOOL;
190   Reset        : BOOL;
191   ModeAuto     : BOOL;
192   ModeManual   : BOOL;
193   JogForward   : BOOL;
194   JogReverse   : BOOL;
195   PartPresent  : BOOL;
196   AlarmAck      : BOOL;
197   LabelReady    : BOOL;
198   ApplyPressure : LREAL;
199 END_VAR
200 VAR_OUTPUT
201  Busy          : BOOL;
202
203 // HMI Mapping
204 StateDisplay := State;
205 CASE State OF
206   0: StatusText := 'Idle';
207   100: StatusText := 'Picking Label';
208   200: StatusText := 'Applying';
209   300: StatusText := 'Complete';
210 ELSE
211   StatusText := 'Unknown';
212 END_CASE
213 Busy := (State <> 0) AND NOT Done;
214 AlarmActive := AlarmLatched;
215 Error := AlarmLatched;
216 ErrorID := AlarmCode;
217
218 // State machine
219 CASE State OF
220   0:
221     LabelVacuum := FALSE;
222     ExtendApplicator := FALSE;
223     Done := FALSE;
224     IF Execute AND PartPresent AND LabelReady AND Active
225       State := 100;
226     END_IF
227   100:
228     LabelVacuum := TRUE;
229     ApplySettle(IN := TRUE, PT := T#300MS);
230     IF ApplySettle.Q THEN
231       ApplySettle(IN := FALSE);
232       State := 200;
233     END_IF
234   200:
235     ExtendApplicator := TRUE;
236     ApplySettle(IN := TRUE, PT := T#400MS);
237     Timeout(IN := TRUE, PT := T#3S);
238     IF Timeout.Q THEN
239       State := 300;
240     END_IF
241   300:
242     State := 0;
243   END_CASE
```

Every station has the same five sections

BECKHOFF

- Every function block seems pretty similar, right?
- Anybody see any problems?

- ▶ **Drift #1** — alarm-ack: edge vs level.
- ▶ **Drift #2** — step numbering: 0/100/200/300 vs 0/10/20/30.
- ▶ **Drift #3** — dead variable: ManualStep in Inspect, never used.

Drift #1 — alarm-ack: edge vs level

BECKHOFF

STRUCTURED TEXT

```
// FB_StationFill – uses R_TRIG (edge-triggered)
fbAckTrig(CLK := AlarmAck);
IF fbAckTrig.Q AND AlarmActive THEN
    AlarmActive := FALSE;
    AlarmCode   := 0;
END_IF

// FB_StationCap – uses level (held-high)
IF AlarmAck AND AlarmActive THEN
    AlarmActive := FALSE;
    AlarmCode   := 0;
END_IF

// They look identical. They behave differently the moment AlarmAck is wired
// to a maintained switch instead of a pushbutton.
```

Drift #2 — step numbering

BECKHOFF

STRUCTURED TEXT

```
// FB_StationFill, Cap, Inspect – clean 0/10/20/30
```

```
CASE State OF
```

```
    0:    // idle
```

```
   10:    // run
```

```
   20:    // complete
```

```
   30:    // fault
```

```
END_CASE
```

```
// FB_StationLabel – copy-pasted from a different machine years ago
```

```
CASE State OF
```

```
    0:    // idle
```

```
   100:   // run
```

```
   200:   // complete
```

```
   300:   // fault
```

```
END_CASE
```

STRUCTURED TEXT

```
// FB_StationInspect – declared, never read, never written
VAR
    State           : INT;
    AlarmActive     : BOOL;
    AlarmCode       : INT;
    ManualStep      : INT;    // <-- nothing in this FB references it
    PartPresent     : BOOL;
    Result          : BOOL;
END_VAR

// Probably copied from a different station's draft 18 months ago.
// Nobody removed it because nobody knew it was safe to remove.
```

“

Anyone here written something that looks like this in the last six months?

— Hold on to that feeling — we're about to make it worse.

CUSTOMER

This is fine. We've shipped 50 machines like this. Why are we changing it?

ANSWER

You're right — it's fine for one machine. We're going to look at what happens when the same change has to land in four files instead of one. If that's never happened to you, this workshop won't change your mind. If it has, you'll recognize the pain.

Goal:

- Add a **ModePause** input ModeAuto and ModeManual
- Every station holds when ModePause is **TRUE**.
- **Exception** — Fill must ignore Pause mid-cycle.

Instructions:

- Open branch release project FillingLine_Inheritance
- Add ModePause to input of every function block
- Spend 5-10 minutes on working through solution

```
git diff stage-1-procedural...stage-1-cr1-applied --stat
```

5 files · +54 -13

-
- ▶ Same 25 lines of guard logic, paid four times (aka cross-cutting)
 - ▶ **Fill's exception** — easy to miss, easy to copy wrong.
 - ▶ Tomorrow's CR pays this cost too. And the next.

} Easy to underestimate what a risk a “simple” change can be

“ Quick check — how many of you copied *Pause* logic into *Label* from *Cap*? From *Fill*? From *Inspect*? Now: check if *FB_StationFill* has a level-based *AlarmAck* or if it has trigger-based *AlarmAck*?

— That's the procedural maintenance trap. Drift you didn't even know was there.

CUSTOMER

My team's careful. We use code review. Drift won't happen.

FAE ANSWER

Code review against duplicated logic catches drift sometimes. Drift you don't catch is the kind that ships. The architecture either enforces consistency or it doesn't — and code review is your last line of defense, not your first.

Goal:

- Flag visual defects **without** stopping the line.
- Camera and reject pre-arm must occur at the same time (**parallel**), not sequentially.
- Learn diff

Instructions:

- Time does not permit attempt to write the code (**expect 60+ lines of code changes**)
- Compare work in the release branch vs work in CR2-applied branch
- **Notice:** Changes are only in FB_StationInspect.

```
git diff stage-1-procedural...stage-1-cr2-applied --stat
```

2 files · +48 -12

- ▶ Inspect's CASE machine rewritten end-to-end.
- ▶ **Parallel coordination buried in CASE** — nothing reusable.
- ▶ The next station that needs parallel pays this cost again.

Goal:

- ► Log cycle data for Fill and Inspect. Cap and Label don't need it.
- ► **Independent work for future to really hammer home the repetitiveness of procedural coding**

CR-3 result — small, but defend it as fine?

BECKHOFF

```
git diff stage-1-procedural...stage-1-cr3-applied --stat
```

2 files · +9 -1

- ▶ Identical buffer code in two places.
- ▶ **Logger evolves in 6 months** — two places to change. Forget one? Drift.
- ▶ Smallest line count. Loses on every other dimension.

Pros	Cons
Simple and familiar Every controls engineer knows this structure	Same change made in every station file No mechanism to enforce consistency
Easy to read Entire station behavior is visible in one file	Copy-paste is the only tool for sharing logic Drift is inevitable
No framework knowledge required	Discipline is the only defense against inconsistency And discipline is bounded
Fast to write for first implementation	Cross-cutting changes ripple across every station file
Easy to debug No inheritance chain to trace	One requirement, five files to touch – and counting

BREAK

15 min

Coffee, bathroom, social conversation.

Stage 2 — Inheritance

60 min

The duplication evaporates.

Then the base class becomes the constraint.

- ▶ **Origin** — Simula 67, Smalltalk 1972; mainstream via C++ / Java.
- ▶ **Mental model** — base encodes shared; children override to specialize.
- ▶ **IEC 61131-3** — inheritance included in 3rd edition (2013)
- ▶ **Promise** — write the common path once.
- ▶ **Catch** — the common path becomes a constraint.

Goal:

- Get familiar with the inheritance sample structure
- Understand OOP keywords: EXTENDS, THIS^, SUPER^
- Understand overriding base class methods
- See a glimpse of SPT Component Base

Instructions:

- Open the release branch of the solution
- Go to the FillingLine_Inheritance Project
- Install project libraries if necessary
- Open FB_StationBase and identify where the common procedural code blocks have been placed
- Compare size of FB_StationFill in procedural vs inheritance projects

- **Mode block — ???**
- **Alarm-ack block — ???**
- **Reset block — ???**
- **HMI mapping — ???**
- **State machine — ???**

- ▶ **Mode block** — Monitoring()
- ▶ **Alarm-ack block** — Monitoring()
- ▶ **Reset block** — Reset()
- ▶ **HMI mapping** — UpdateHmiStatus()
- ▶ **State machine** — ???

- ▶ **Mode block** — Monitoring()
- ▶ **Alarm-ack block** — Monitoring()
- ▶ **Reset block** — Reset()
- ▶ **HMI mapping** — UpdateHmiStatus()
- ▶ **State machine** — ExecuteSequence() of derived FB overrides base

Inheritance mechanics — four things to recognize

BECKHOFF

```
// 1. EXTENDS – child declares parent
FUNCTION_BLOCK FB_StationFill EXTENDS FB_StationBase

// 2. SUPER^ – explicit base call
METHOD PROTECTED CyclicLogic
    SUPER^.CyclicLogic();    // run the base's mode/ack/reset
    THIS^.ExecuteSequence(); // then run my sequence

// 3. Method override – child's version replaces base's
METHOD PROTECTED ExecuteSequence
    CASE State OF
        0: IF Mode.AllowRun THEN State := 10; END_IF
        10: // ... fill logic
        20: State := 0;
    END_CASE

// 4. FB_init – construction-time parameters
METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains : BOOL;
    bInCopyCode  : BOOL;
    Name         : STRING(20);
END_VAR
SUPER^.FB_init(bInitRetains, bInCopyCode);
THIS^.Name := Name;
```

- ▶ **FB_StationFill** — 107 lines → 67. **42% smaller**
- ▶ **Drift #1 fixed** — base normalizes alarm-ack to R_TRIG.
- ▶ **Drift #2 fixed** — Label normalized to 0/10/20/30.
- ▶ **Drift #3 fixed** — dead ManualStep doesn't propagate.

- Where is Monitoring() called?

CUSTOMER

My engineers don't know inheritance. Now they can't read my code.

FAE ANSWER

That's a valid concern. Inheritance has a learning cost. The win is consistency enforcement — once your team learns it, drift goes away. The trade-off is real; you pick whichever fits your team's experience curve.

Goal:

- Add a **ModePause** input mimicking ModeAuto and ModeManual
- Every station holds when ModePause is **TRUE**.
- **Exception** — Fill must ignore Pause mid-cycle.
- **Use inheritance**

Instructions:

- Start from release branch in FillingLine_Inheritance Project
- **Option A:** add AllowPause() on the base; add AllowPause() to Fill to override base method
- **Option B:** add Monitoring() to Fill Station to override base method; use SUPER^.Monitoring() to include base code
- Spend 5-10 minutes writing code

```
git diff stage-2-inheritance...stage-2-cr1-applied --stat
```

3 files · +? -?

► Inheritance vs Procedural

- Majority of code changes reside in base class
- Cap, Inspect, Label unchanged!

► Option A: Pollution lives on the base

- AllowPause() is included in every derived FB just to accommodate one exception for Fill.

► Option B: Base code logic duplicated

- overriding Monitoring() requires keeping track of duplicate code in multiple locations

► Inheritance didn't eliminate the cost. It relocated it.

CR-2 — breaking the inheritance chain

25 min ·

Same as Stage 1, applied via inheritance.

Watch the override fight the base.

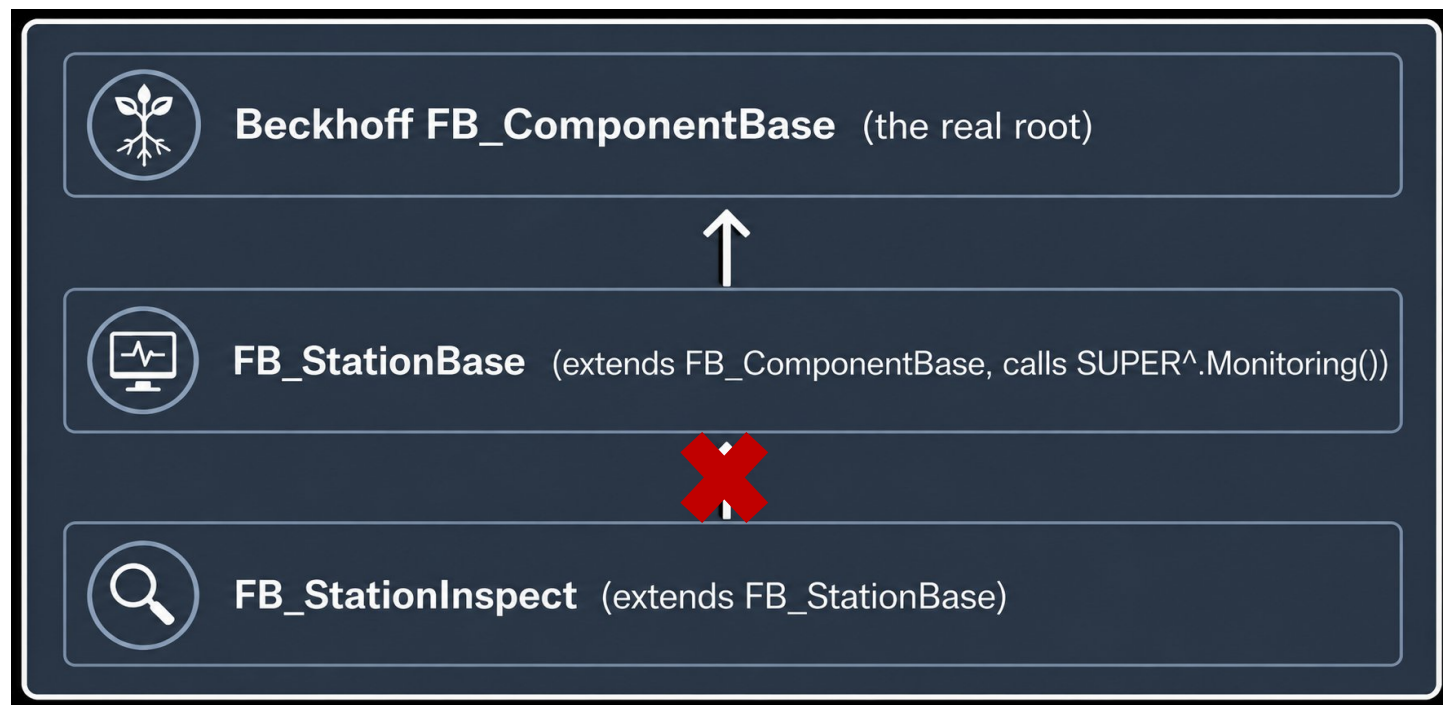
Goal:

- Flag visual defects **without** stopping the line.
- Camera and reject pre-arm must occur at the same time (**parallel**), not sequentially.
- **Use Inheritance**

Instructions:

- Compare work in the release branch vs work in CR2-applied
- **Notice:** Changes is only in FB_StationInspect methods
 - ExecuteSequence()
 - Monitoring() overrides the base class – SUPER^.Monitoring() is not called, instead programmer copied contents of base method.

- Skipping SUPER^ severs the inheritance chain silently – no compiler warning
- Base and root class improvements no longer flow to the derived class
- Duplicated logic must be maintained in two places indefinitely
- A single missed update creates subtle, hard-to-find bugs
- **Inheritance only saves work if the chain stays intact**



```
git diff stage-2-inheritance...stage-2-cr2-applied --stat
```

2 files · +121 -48

- ▶ Inspect overrides Monitoring without calling SUPER.
- ▶ ~80 lines reimplement mode + ack from scratch.
- ▶ Every future base change is now "did we update Inspect?"

“ *The base class that helped you in Stage 1 → Stage 2 has now become the constraint. Inspect can't extend the base. It has to fight the base.*

— This isn't your fault. It's a property of the design. Single inheritance is all-or-nothing.

CUSTOMER

This is fine — I just need to remember to mirror base changes into Inspect's override.

FAE ANSWER

That's a code-review burden you'll carry forever. Every base change is now a question of 'did we update Inspect?' Sometimes you'll forget. The architecture isn't enforcing the consistency anymore — your team's discipline is. Discipline is bounded; architecture is unbounded.

Goal:

- Log cycle data for Fill and Inspect. Cap and Label don't need it.

Instructions:

- Open CR3-applied branch and go to FillingLine_Inheritance
- Find buffer in FB_StationBase
- Find added methods in FB_StationBase
- Find overriding methods in derived FBs

```
git diff stage-2-inheritance...stage-2-cr3-applied --stat
```

3 files · +64 -25

- ▶ LogBuffer + LoggingEnabled added to the base.
- ▶ Cap and Label inherit storage they never use.
- ▶ Cognitive cost compounds even when memory cost doesn't.

Block 2 debrief — Pros and Cons

BECKHOFF

Pros	Cons
Common code written once, inherited by all	Base accumulates features all children inherit whether they need them or not
Architecture enforces consistency – no discipline required	No way to modify a part of an inherited method – it's all or nothing
Cross-cutting changes touch one file instead of four	Skipping SUPER^ severs the entire framework chain (consequence of above)
Stage 1 drift eliminated	The base that helped in CR-1 becomes the constraint in CR-2

BREAK

15 min

The longer of the two breaks if you can swing it. Block 3's payoff lands harder if the room comes back fresh.

Stage 3 — Composition

90 min

Build the line from interface-typed building blocks.

The composition paradigm

BECKHOFF

- **Origin** — Gang of Four 1994; SOLID 2000s.
- **Mental model** — small focused FBs talk through interfaces (stations see the interface, never the FB that implements it)
- **IEC 61131-3** — INTERFACE, IMPLEMENTS, FB references typed by interface, FB_init for dependency injection (DI)
- **Promise** — variation by injection
- **Catch** — more files, more concepts, more vocabulary

Goal:

- Get familiar with the composition sample structure
- Understand OOP terms INTERFACE, IMPLEMENTS, constructor
- Understand dependency injection (DI) with FB_Init

Instructions:

- Open the release branch and go to the FillingLine_Composition Project
- Open the POU folder and find the Interfaces, Building Blocks, and Stations
- Note:
 - No base class!

- **Interface**

- A contract between a building block and everything that uses it
- (x5) I_AlarmHandler, I_DataLogger, I_HmiReportable, I_Sequenceable, I_StationCore

- **Building block**

- Small, focused FB that serves only one purpose
- (x4) alarm handlers, loggers, mode manager, sequencers

- **Station**

- Coordinates the building blocks
- (x4) Fill, Cap, Label, Inspect

- **MAIN** – wires the dependency graph

TWINCAT INTERFACE

```
// I_AlarmHandler.TcPOU – a contract. No implementation.

INTERFACE I_AlarmHandler

PROPERTY IsActive : BOOL    // is an alarm currently latched?
PROPERTY StopsLine : BOOL   // should this alarm stop the line?
                             // (TRUE for line faults, FALSE for quality flags)

METHOD Raise
VAR_INPUT
    Code    : INT;
    Text    : STRING(80);
END_VAR

METHOD Acknowledge
END_METHOD

// Anyone who implements this satisfies the contract.
// Stations depend on I_AlarmHandler – they never know which implementation
// they have. That's the seam where CR-2's swap will happen.
```

Why use the term contract?

Why use the term contract?

	Points To	<u>Compiler Enforced</u>
POINTER TO BOOL	A memory address that may or may not contain a Boolean value	No
REFERENCE TO FB_myFunctionBlock	A specific FB type (FB_myFunctionBlock)	Yes – but locked to that type
I_myInterface	Anything (e.g. FB, ITF) that honors the contract (e.g. contains methods, properties, etc)	Yes – and flexible

FB_ALARMHANDLER_LINEFAULT

- ▶ **StopsLine** — TRUE.
- ▶ **Raise()** — latches, blocks downstream.
- ▶ **Use for** — real faults that should stop the line.
- ▶ **Tells the station** — "halt."

FB_ALARMHANDLER_QUALITYFLAG

- ▶ **StopsLine** — FALSE.
- ▶ **Raise()** — sets Code/Text, does NOT latch.
- ▶ **Use for** — quality defects that feed sortation.
- ▶ **Tells the station** — "note this part; keep running."

Same Contract, Different Behavior

Stations — composition of smaller pieces

BECKHOFF

```
FUNCTION_BLOCK FB_StationFill IMPLEMENTS I_HmiReportable, I_Sequenceable

VAR
    Mode      : REFERENCE TO FB_ModeManager;
    Alarm      : I_AlarmHandler;           // <-- depends on the INTERFACE
    Logger      : I_DataLogger;           // <-- same
    Seq        : FB_StepSequencer;
END_VAR

METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains : BOOL;
    bInCopyCode   : BOOL;
    ModeRef       : REFERENCE TO FB_ModeManager;
    AlarmHandler  : I_AlarmHandler;       // <-- injected at construction
    DataLogger    : I_DataLogger;       // <-- injected at construction
END_VAR

THIS^.Mode REF= ModeRef;
THIS^.Alarm := AlarmHandler;
THIS^.Logger := DataLogger;

// The station never constructs its dependencies. It receives them.
// That's dependency injection – the seam that makes CR-2 trivial.
```

- Variables typed by interface
- Dependencies injected at construction via FB_Init
- MAIN decides what the station gets

Inheritance	Composition
Station is a base class	Station has building blocks
Behavior comes from the parent	Behavior comes from the pieces
EXTENDS	FB_Init parameters
One rigid hierarchy	Freely assembled combinations

IS VS HAS

MAIN — the construction is the architecture

BECKHOFF

STRUCTURED TEXT

```
PROGRAM MAIN
VAR
    Mode          : FB_ModeManager;
    LineFault      : FB_AlarmHandler_LineFault;
    QualityFlag    : FB_AlarmHandler_QualityFlag;    // <-- both available
    CycleLogger    : FB_CycleDataLogger;

    Fill          : FB_StationFill(
        ModeRef      := Mode,
        AlarmHandler  := LineFault,                // <-- swap point
        DataLogger    := CycleLogger);

    Cap           : FB_StationCap(
        ModeRef      := Mode,
        AlarmHandler  := LineFault);

    Label         : FB_StationLabel(
        ModeRef      := Mode,
        AlarmHandler  := LineFault);

    Inspect       : FB_StationInspect(
        ModeRef      := Mode,
        AlarmHandler  := LineFault);                // <-- this line
END_VAR
// Read MAIN top-to-bottom: it IS the wiring diagram for the line.
```

- ▶ **S** — Single Responsibility — one job per FB.
- ▶ **O** — Open/Closed — add new alarm strategy, existing code untouched.
- ▶ **L** — Liskov Substitution — any I_AlarmHandler works anywhere.
- ▶ **I** — Interface Segregation — HMI sees I_HmiReportable only.
- ▶ **D** — Dependency Inversion — stations depend on I_AlarmHandler.

CUSTOMER

Eighteen files for a 4-station machine? You've got to be kidding.

FAE ANSWER

You're right — for 4 stations, this is overkill. The architecture pays back when the codebase grows. If you're shipping one machine and walking away, Stage 1 is the right answer. If you're building a product family, Stage 3 is the prerequisite for sane long-term maintenance.

```
git diff stage-3-composition...stage-3-cr1-applied --stat
```

2 files · +19 -6

- ▶ Just modify FB_ModeManager. Stations are not touched.
- ▶ **Fill's mid-cycle exception** — automatic.
- ▶ The abstraction already encoded the right behavior.

CR-2 — the headline swap demo

20 min · the workshop's emotional payoff

The swap — one line in MAIN

BECKHOFF

```
// MAIN.TcPOU – find this line:

Inspect      : FB_StationInspect(
                ModeRef      := Mode,
                AlarmHandler  := LineFault);    // <-- this

// Change to:

Inspect      : FB_StationInspect(
                ModeRef      := Mode,
                AlarmHandler  := QualityFlag);  // <-- this

// Save. Build.
//
// Inspect's behavior just changed: alarms are now non-latching,
// the line keeps running on defect. NO INSPECT CODE WAS MODIFIED.
```

“
Stage 1 CR-2: 92 lines of inline rewrite.
Stage 2 CR-2: 121 lines of override fight.
Stage 3 CR-2: ONE LINE.

— Same requirements, three architectures, different blast radius.

- ▶ Compose FB_ParallelSequencer alongside FB_StepSequencer.
- ▶ Configure for 2 branches in FB_init. Rework state 10.
- ▶ Composition, not rewriting.
- ▶ **Final diff** — 2 files, ~30 lines added.

```
git diff stage-3-composition...stage-3-cr3-applied --stat
```

3 files · +32 -11

- ▶ Add I_DataLogger field + FB_init param to Fill and Inspect only.
- ▶ **Cap and Label keep their 4-arg signature** — DON'T touch them.
- ▶ **Logger swap (in-memory → file → DB)** — replace one FB. Stations don't change.

- ▶ **git switch stage-3-broken** — Pause via per-station guards.
- ▶ Compiles. Runs. Halts Fill mid-cycle.
- ▶ The right architecture nudges, doesn't force.
- ▶ Right answer: 1-file diff. Wrong answer: 5-file diff.

Scoreboard, discussion, take-home

20 min · plan to overshoot

The numbers from the diffs go on the board.

When does each stage fit your real workload?

	Stage 1 — Procedural	Stage 2 — Inheritance	Stage 3 — Composition
CR-1 Pause mode	5 files +50 -13	3 files +65 -48	2 files +19 -6
CR-2 Inspect quality + parallel	2 files +92 -55	2 files +121 -48	1 line in MAIN + ~3 internal
CR-3 Selective logging	2 files +13 -3	3 files +64 -25	3 files +32 -11

Stage 3 wins on every CR. The shape of the win matters more than the line count.

- ▶ **Stage 1** — every CR touches every station.
- ▶ **Stage 2** — base + affected children. Until pollution.
- ▶ **Stage 3** — building block touched. Others untouched.

- ▶ **Stage 1 / 2 stations** — entangled with I/O. Hard to test.
- ▶ **Stage 3 stations** — accept dependencies at construction.
- ▶ **Today** — PlcTestSuite + DI + mocks → deterministic tests.
- ▶ **Coming** — Beckhoff-native framework. TcUnit also exists. Both heavier lifts than PlcTestSuite.
- ▶ If your shop wants CI for PLC code, this is the prerequisite.

Which stage for which shop?

BECKHOFF

Stage 1 — Procedural is right for

- ▶ Single-machine integrators
- ▶ Panel shops
- ▶ One-engineer teams
- ▶ Short-lifespan code
- ▶ Ship-and-walk-away contracts

Stage 2 — Inheritance is right for

- ▶ Homogeneous machine families
- ▶ Mid-sized teams
- ▶ Long-lifespan, medium variation
- ▶ Teams comfortable with EXTENDS / SUPER

Stage 3 — Composition is right for

- ▶ OEMs building product families
- ▶ Multi-engineer teams
- ▶ Multi-customer variation
- ▶ Long-lifespan codebases
- ▶ Shops moving toward CI / TDD for PLC

- ▶ You'll go back. Someone will say "this is overkill" or "just give me the procedural version."
- ▶ Pick three pushbacks. Roleplay.
- ▶ The answer isn't "composition is always right."
- ▶ Discipline is bounded; architecture is unbounded.

- ▶ **Self-paced refresher** — ./serve-docs.sh, walk the stages.
- ▶ **Teaching mode** — run a 1-2 hour version for your team.
- ▶ **Deep end** — PlcTestSuite on Stage 3 vs Stage 1. Visceral.
- ▶ Send me one specific machine this material applies to.

- ▶ **Send each attendee a brief follow-up** — one specific recommendation tied to their team.
- ▶ **Sarah** — motion controllers. Try the I_AlarmHandler swap on test rigs.
- ▶ **Bob** — single-machine codebase. Stage 1 is fine. Try the CR-2 swap exercise on your next vendor swap.
- ▶ Generic advice fades. Specific advice sticks.

FROM PROCEDURAL TO SOLID • CLOSING

Become believers, then teachers

The believer part comes from doing the workshop. You did that today.

The teacher part requires three more things: run a shorter version yourself within a month; adapt the script to your audience; track customer pushback you hear that isn't in this material — send it back.

The deepest measure of this workshop's success is when an FAE who attended teaches it, and their attendees recognize themselves in Stage 1's drift the way you recognized yourselves today.

1. Thoughts?
2. Objections?
3. Cost /Benefit
4. When and where to recommend each
5. Do not get drawn into the emotional dogmatic approach all are valid paradigms for machine architecture.

THANK YOU FOR YOUR TIME

Facilitator reference (skip in live presentation)

reference · not for projection

Pacing levers. PlcTestSuite demo script. Materials. Shorter versions.

Pacing levers — what to cut, what to add

BECKHOFF

IF YOU'RE 15+ MIN BEHIND

- ▶ **Block 2 CR-3** — cut hands-on. Just show the diff.
- ▶ **Block 1 debrief** — tighten. Show matrix; let audience scan.
- ▶ **Block 3 CR-2** — demo the swap, skip the parallel sequencer.
- ▶ **Block 4 4A** — compress to a 2-min walk-through.

IF YOU'RE AHEAD

- ▶ **Block 4 4B** — extend customer-conversation rehearsal.
- ▶ **Show patterns.md** — name Strategy / Template Method / DI.
- ▶ **Demo stage-3-broken** — why wrong even though it compiles.
- ▶ **PlcTestSuite demo** — most powerful extension. See next slide.

Bonus — PlcTestSuite demo (only if class finishes early)

BECKHOFF

- ▶ **When** — 20+ min ahead at Block 4, runtime available, room is engaged.
- ▶ **Branch** — stage-3-complete-tests. Library — PlcTestSuite (separate install).
- ▶ **2 min** — read 'Why composition is testable' from testing.md.
- ▶ **5 min** — show Test_QualityFlag_StopsLineIsFalse. Six lines = Liskov.
- ▶ **5 min** — show Test_FillStation_HappyPath_CompletesCycle. The DI seam IS the testing seam.
- ▶ **5 min** — if you have a runtime: build, login, RunTests := TRUE, show TEST_Result.xml.
- ▶ **Why PlcTestSuite** — TcUnit and the upcoming Beckhoff-native framework are heavier installs.

DAY-OF

- ▶ Repo cloned, all branches fetched.
- ▶ TwinCAT XAE 3.1.4026+, two instances open.
- ▶ Browser with the scoreboard bookmarked.
- ▶ **Local docs site (./serve-docs.sh)** — wifi backup.
- ▶ Whiteboard or large sticky notes for the physical scoreboard.
- ▶ Timer for the timed exercises.

DAY-AFTER

- ▶ Send each attendee a personal recommendation.
- ▶ Send the repo URL and scoreboard compare links.
- ▶ Solicit feedback in the workshop's chat / channel.
- ▶ **Optional** — only if you ran testing demo: PlcTestSuite link.

Shorter versions — when 4 hours isn't on the table

BECKHOFF

- ▶ **30 min** — "the one demo." Stage 3 CR-2 swap only.
- ▶ **1 hr** — "the methodology compare." Stage 1 + Stage 3 CR-1; skip Stage 2.
- ▶ **2 hr** — "the workshop, compressed." Cut CR-3 in every stage.
- ▶ **4 hr** — "the full workshop." This deck.